

GMXBUILDER User Manual

Document version	1.0.0
Software	GMXBUILDER 1.0.0
Author	Haochen Yang
Release date	2026-10-08
Status	Public release

This manual describes validated preparation of starting systems. Production simulation still requires minimization, sufficient equilibration, repeat sampling, and scientific review.

Contents

Select a section title or page number to follow the document link.

Change log	3
1. Introduction	4
1.1 Available workflows	4
1.2 Check, Viewer, and Build	4
1.3 Scientific responsibility	4
1.3.1 Free protein termini	4
1.3.2 Water models	5
1.4 Supported input bounds	5
2. Installation and service startup	6
2.1 Bootstrap requirements	6
2.2 Unattended local installation	6
2.3 Manual installation	7
2.3.1 Install runtimes, software, and assets	7
2.3.2 Configure runtime paths for the current shell	7
2.4 Start and inspect the service	7
3. Web interface	9
3.1 Task ID and browser routes	9
3.2 Bilayer Builder	9
Step 1 — Input Structure	9
Step 2 — Force Field	9
Step 3 — Structure Processing	9
Step 4 — Protein Orientation	9
Step 5 — Membrane Builder	9
Step 6 — Solvent & Box	10
Step 7 — Ions	10
Step 8 — Final Structure Review	10
Step 9 — Simulation Parameters and Build	10
3.3 Pure Bilayer System	10
3.4 Solvator	10
3.5 Martini 3 Builders	11
4. Command-line interface	12
4.1 Discover commands and capabilities	12
4.2 YAML atomistic build	12
4.3 Martini 3 example	13
5. HTTP API	14
5.1 Discovery and error handling	14
5.1.1 Managed asynchronous operations	14

5.2 Minimal Solvator sequence	15
5.2.1 Final structure confirmation	15
5.3 Build, queue, and download	15
6. Output package and execution	16
6.1 Package layout	16
6.2 Run the generated package	16
7. Troubleshooting	17
7.1 A lipid is unavailable	17
7.2 Bilayer orientation or packing looks wrong	17
7.3 Automatic protein orientation is implausible	17
7.4 Water layers appear unequal	17
7.5 Ions cluster or water is missing from the Viewer	17
7.6 ZIP lacks MDP files or run_md.sh	17
7.7 A Task cannot be resumed	17
7.8 A pH is refused, or a build warns about the termini	17
7.9 A water model is refused as expert-unvalidated	17
7.10 A size limit is reported	18
7.11 Automatic installation cannot build GROMACS	18
8. Getting help	19
Appendix A. Deployment configuration	20

Change log

Version	Date	Contents
1.0.0	2026-10-08	Stable release; the V4 lipid library, five workflows, asynchronous API, final review, installation requirements and output layout

This manual applies to software 1.0.0. The V4 lipid library supplies 37 accepted combinations, 134,000 initialization conformers and 67 GAFF2/AM1-BCC parameter caches. Prepared parameters do not imply conformer acceptance; initialization acceptance does not certify membrane equilibrium. See the [release support matrix](#).

1. Introduction

GMXBUILDER prepares GROMACS input packages for atomistic membrane-protein, pure-bilayer, and aqueous systems, and for supported Martini 3 coarse-grained systems. The Web, CLI, and HTTP API use the same validation rules.

1.1 Available workflows

Workflow	Purpose
Bilayer Builder	Process and orient a membrane protein, build a bilayer, add water and ions
Pure Bilayer System	Build a protein-free dry or solvated bilayer
Solvator	Build aqueous protein, canonical linear DNA/RNA, protein-nucleic-acid, and compatible ligand systems
Martini 3 Bilayer Builder	Build a flat membrane or protein-membrane CG system with explicit leaflet counts/compositions
Martini 3 Solvent Builder	Build a standard-protein aqueous CG system

Cards that are disabled in the installed interface are not current product capabilities. Query the application instead of relying on a static molecule count.

1.2 Check, Viewer, and Build

Every Check validates the current step and saves one authoritative task checkpoint. The Viewer reloads the saved coordinates. Later steps consume that checkpoint, so a changed upstream setting invalidates downstream results.

Final Build does not reconstruct the membrane, water, or ions. It assigns the topology and packages the last confirmed coordinates with the selected MDP stages and launcher script.

1.3 Scientific responsibility

A successful build establishes structural and topology consistency under the implemented checks. It does not establish a correct biological orientation, protonation state, ligand chemistry, equilibrium phase, or converged production trajectory. Review the [scientific limitations](#) before simulation.

Two boundaries are reported during a build rather than left to be discovered afterwards.

1.3.1 Free protein termini

Uncapped termini are built as the canonical charged templates, NH₃⁺ and COO⁻; no neutral terminal microstate is implemented. The bounds follow from the model pK_a values rather than being chosen. Between pH 4.45 and 7.05 both canonical states are at least 90% populated and the build is silent. Between pH 3.50 and 8.00 the canonical state is still the majority species: the build proceeds and reports the actual population, because molecular dynamics assigns one discrete state per titratable group and the convention is to assign the dominant one. Beyond the pK_a values the canonical form would be the minority species, and the build stops.

These model pK_a values are construction-policy approximations, not measured values for a particular protein. ACE/FOR/NME caps change molecular identity. Select them only when the experimental construct or modeling objective calls for that chemistry; they are not a general workaround for unsupported pH or neutral free-terminal states.

1.3.2 Water models

A force-field and water-model pair is classified before use. A pair that is merely bundled, rather than being the force field's default or covered by regression tests, is reported as expert-unvalidated and requires `allow_unvalidated_water_model: true` to proceed. The classification and its policy version are recorded in the exported system metadata.

1.4 Supported input bounds

These are limits of the implemented construction, not statements about scientific validity. Exceeding one is reported with the offending value.

Input	Bound
Starting lipids per leaflet (AA); exact count (Martini)	64 to 5000
Bilayer or solvation box, per axis	100 nm
Solvation box volume	50000 nm ³
Side-chain pKa estimation	pH 1.0 to 13.0
Ligand atoms for GAFF2 or CGenFF	2048
Uploaded structure	32 MB, 250000 atoms (see below)

Upload limits are configurable through the environment and are clamped to a hard maximum rather than silently reduced: a request above the maximum is granted the maximum, and the refusal is written to the service log. A request below the minimum is treated the same way.

2. Installation and service startup

2.1 Bootstrap requirements

1. Linux x86-64 and Python 3.10 or later.
2. Git, CMake, a C++17 compiler, and Python's `venv` module.
3. Internet access during first installation.
4. The NVIDIA CUDA toolkit, including `nvcc`, only when the managed GROMACS runtime should be built with CUDA acceleration.

The installer manages the required GROMACS runtime, Python environment, GAFF2/AM1-BCC tools, force-field data, and verified prebuilt lipid assets. Users therefore do not need to install GROMACS, AmberTools, ACPYPE, Open Babel, Martini 3 data, or Python packages separately. The complete managed GAFF lock is verified for Linux x86-64; other platforms require an independently validated scientific runtime. Managed Web operation additionally requires a systemd user manager, cgroup v2, Landlock ABI 3+, FUSE3, `fusermount3`, `/dev/fuse`, `libfuse3-dev` and `pkg-config`. Once the system prerequisites are installed, the application and scientific runtimes can be installed under the user account. See the [resource setup guide](#).

2.2 Unattended local installation

```
git clone https://github.com/AkaseToshiyuki/GMX-BUILDER.git
cd GMX-BUILDER
./install-local.sh
```

With no arguments, the installer is unattended: it binds to loopback on port 7788, assigns half of the detected CPU cores, derives a compatible queue size, and starts the user service. It reuses an explicitly selected or PATH-visible GROMACS 2026.0-or-newer executable. Otherwise it downloads the official GROMACS 2026.3 source archive, verifies its pinned SHA-256 digest, and builds a private runtime under the user's GMXBUILDER data directory. CUDA is enabled when `nvcc` is available; otherwise a complete CPU runtime is built.

The same installation creates a private GAFF2/AM1-BCC environment containing pinned AmberTools, ACPYPE, and Open Babel packages from conda-forge. It also downloads every separately distributed force-field port listed in [scripts/external_assets.json](#) and the V4 lipid archive (bundle revision 7, schema 4) from manifest-pinned HTTPS locations, verifies their digests, installs the locked Python environment, and populates the user cache. Git LFS, a GitHub token, and root access are not required. Run `./install-local.sh --help` for command-line overrides or `./install-local.sh --interactive` for prompts. One Task remains strictly serial; safe computation inside its current step may use multiple threads.

Select a compatible existing executable explicitly when desired:

```
./install-local.sh --gmx-bin /opt/gromacs/bin/gmx
```

Force a CPU-only managed GROMACS build even when CUDA is installed:

```
GMXBUILDER_GROMACS_FORCE_CPU=1 ./install-local.sh
```

2.3 Manual installation

2.3.1 Install runtimes, software, and assets

```
python3 scripts/install_gromacs.py
python3 scripts/install_gaff_runtime.py
python3 scripts/install_external_assets.py
python3 scripts/fetch_prebuilt_assets.py
uv sync --frozen --no-dev
source .venv/bin/activate

gmxbuilder --version
gmxbuilder prebuilt-assets status
gmxbuilder prebuilt-assets install
```

2.3.2 Configure runtime paths for the current shell

Add the managed GROMACS executable to the current shell when using the manual sequence, or export its absolute path as **GMX_BIN**:

```
export GMX_BIN="$HOME/.local/share/gmxbuilder/runtime/gromacs-2026.3/bin/gmx"
export GMXBUILDER_GAFF_ENV="$HOME/.local/share/gmxbuilder/gaff-env"
```

The download bootstrap and asset installer verify the archive checksum, strict-library schema, and every included conformer-library entry before it writes the cache. A release contains only the force-field/lipid combinations that passed the production quality gates when it was built; other compatible combinations remain visibly unavailable. Installation does not overwrite a newer existing cache.

2.4 Start and inspect the service

```
gmxbuilder serve
```

Open <http://127.0.0.1:7788/>. Resource limits can be selected explicitly:

```
gmxbuilder serve \
--host 127.0.0.1 --port 7788 \
--cpu-cores 24 --task-threads 8 \
--max-builds 3 --gpu-count 1
```

--task-threads must divide **--cpu-cores**. Set **CUDA_VISIBLE_DEVICES** before startup to expose a selected GPU subset or order. Use **--gpu-count 0** for a CPU-only service.

Useful checks:

```
curl -fsS http://127.0.0.1:7788/health
systemctl --user status gmxbuilder.service
```

The default listener is **127.0.0.1:7788**. In **local** or **trusted-lan** mode, a non-loopback listener without global authentication requires explicit **--allow-unsafe-deployment**. Public hosting has two separate modes: **public** requires global authentication, while **public-anonymous** requires managed resource isolation. Both require a configured trusted TLS proxy and explicit HTTPS origins. See Appendix A.

The IPv4 and IPv6 wildcards are covered by the same rule. Selecting a wildcard is not consent to itself — it is the broadest possible exposure and the one most often reached by accident — so the server refuses to start without the opt-in:

```
gmxbuilder serve --host 0.0.0.0 --port 7788 --allow-unsafe-deployment
gmxbuilder serve --host 192.0.2.10 --allow-unsafe-deployment
GMXBUILDER_ALLOW_UNSAFE_DEPLOYMENT=1 \
gmxbuilder serve --host 192.0.2.10
```

The installer enforces the same requirement and persists the choice in the generated local-service runner:

```
./install-local.sh --bind-host 0.0.0.0 --allow-unsafe-deployment
./install-local.sh --bind-host 192.0.2.10 \
--deployment-mode local --allow-unsafe-deployment
```

These listeners have no end-user login in [local](#) or [trusted-lan](#) mode. Keep them behind a private-network firewall and never expose them directly to the Internet. [GMXBUILDER_DEPLOYMENT_MODE=public](#) is not relaxed by the unsafe opt-in: public mode still requires strong global authentication, HTTPS origins, and a trusted TLS reverse proxy as specified in Appendix A. The separate [public-anonymous](#) mode cannot use the unsafe opt-in to bypass resource isolation.

3. Web interface

3.1 Task ID and browser routes

After upload or task creation, save the displayed Task ID using the copy button. Browser routes contain only the workflow and step, for example:

```
/BilayerBuilder/Step3  
/Solvator/Step2
```

The Task ID is not placed in the URL. Refreshing the same tab restores its saved task and server checkpoints. A step URL without a saved task opens the workflow at its first step; unsubmitted edits are not checkpoints. On another tab or device, enter the saved Task ID on the home page to resume or retrieve a completed package. Treat the Task ID as a private access capability.

Tasks expire according to the server retention policy. Task-private uploads, including custom lipid parameters, expire with the Task.

3.2 Bilayer Builder

Step 1 — Input Structure

Upload PDB, mmCIF, or a supported gzip-compressed form. Review chains, retained small molecules, alternate locations, missing heavy atoms, modified residues, and warnings. Deselecting or renaming a component changes the saved input and requires another Check.

Step 2 — Force Field

Select one coherent protein, membrane, ligand, and water-model combination. The UI disables incompatible combinations and the server repeats the check. Amber membranes prefer covered Lipid21 species; mixed Lipid21/GAFF2 models require accepted guest conformers and their recorded host contract. Whole-GAFF2 is an explicit alternative. CHARMM systems use compatible CHARMM lipids and CHARMM-family ligand parameters.

For GAFF2 ligands, confirm the integer net charge for the intended protonation state. CHARMM ligands default to local chemical identification and complete templates. Restricted experimental assignment requires explicit opt-in; matching CGenFF MOL2 and STR import is also available. See the [CHARMM-compatible ligand guide](#).

Step 3 — Structure Processing

Compute protonation assignments at the selected pH, then review termini, modifications, and crosslinks. PROPKA is a static-structure suggestion, not constant-pH MD. Only force-field-specific atom-complete modifications can be selected. Unsupported chemistry remains visible and blocks export.

Step 4 — Protein Orientation

Automatic orientation is an aid, not a biological annotation. Review the grey membrane-interface planes and all extracellular, intracellular, and transmembrane regions. Manual adjustments are saved by Check and used by the next step.

Step 5 — Membrane Builder

Choose upper and lower leaflet compositions and a starting count (64–5000). Check computes one common XY box from the two leaflet APL targets, their own protein footprints within the membrane, and the required protein XY padding. It then fills each leaflet's available area separately. Actual upper and lower counts can differ, even with equal compositions, and can exceed the starting count. A result needing more than 5000 lipids in either leaflet is refused.

Before Check, counts are estimates. After Check, the page reports actual upper/lower totals and species counts from the saved structure, also when a task is resumed. Editing the inputs invalidates those results. These areas are construction targets, not evidence of equilibrium APL or membrane stability. Every lipid must be supported by the selected backend and have a valid conformer. Review leaflet orientation, headgroups facing solvent, tails facing the bilayer core, packing around the protein, and the quality report.

Step 6 — Solvent & Box

Z padding describes the requested water thickness outside the membrane interfaces; XY is inherited from the membrane. Check the box, water layers, atom counts, and that the membrane does not cross a periodic boundary.

Step 7 — Ions

Choose ion species, concentration, neutralization, exclusion radius, and placement method. Seeded uniform random replacement is the validated and recommended default and replaces complete waters at their oxygen coordinates. The formal-charge-ranked and dimensionless Metropolis site-optimization modes also replace complete waters, but are explicitly experimental heuristics; they are not equilibrium ion sampling and must not be interpreted as a predicted ion atmosphere. Click Check Ion Counts, then continue to Final Structure Review.

Step 8 — Final Structure Review

Inspect the saved protein, lipids, water, ions and periodic box in the final Viewer. Component counts come from the checkpoint. Loading reports preparation, download, parsing and drawing stages; confirmation stays disabled until the full scene is drawn. Display omits hydrogens only; exported coordinates are unchanged. Use component toggles to inspect the structure and Retry viewer after a loading error. Confirm Simulation System binds your approval to this checkpoint. Rechecking an upstream step requires another review. This separate review also applies to Solvator, dry Pure Bilayer systems and both Martini 3 workflows.

Step 9 — Simulation Parameters and Build

Enable only the equilibration and production stages you want. Each stage owns its temperature, coupling, cutoffs, restraints, output intervals, COM removal, and duration. Hardware settings affect [run_md.sh](#), not MDP physics. Build packages the confirmed ion checkpoint and does not rerun coordinate generation.

3.3 Pure Bilayer System

This workflow starts with force-field and lipid selection. It uses an independent implementation of membrane, solvation, ions, and export. Clear solvation to export a dry bilayer; the dry package intentionally omits water, ions, MDP files, and [run_md.sh](#).

3.4 Solvator

Solvator omits membrane construction and orientation. Box padding is measured from the full retained solute in all directions and pressure coupling defaults to isotropic.

Canonical linear DNA/RNA is displayed as polymer chains, not small molecules. Select CHARMM36m or installed Amber14SB + OL24. Structure Processing uses native GROMACS residue data for the selected force field to construct hydrogens, 5'/3' termini, polymer bonds, and exact charge. Modified, circular, or broken nucleic-acid chains are blocked. This operation replaces uploaded nucleic-acid coordinates with the hydrogen-complete [pdb2gmx](#) output. Review every nucleic-acid chain in the Step 3 viewer before continuing.

3.5 Martini 3 Builders

Martini 3 uses two independent entries: Martini 3 Bilayer Builder and Martini 3 Solvent Builder. There is no environment selector inside either workflow. The bilayer workflow accepts an exact lipid count per leaflet, symmetric/asymmetric compositions, optional standard protein mapping, an independent PPM-like membrane-orientation Check, regular W water and ions. The installed official PC/PE/PG/PS/SM/sterol topology set is filtered to molecules that the pinned geometry builder can construct and the structural gates can identify. The solvent workflow requires a standard protein. Ligands, PTMs, glycans, nucleic acids, custom CG molecules, curved surfaces, and backmapping are not silently discarded; they are unavailable.

The periodic box is derived automatically from the positioned CG envelope, solvent padding and, for bilayers, a conservative construction area plus the explicit leaflet count. The construction area is an initial packing value, not an equilibrium APL; NPT equilibration relaxes the periodic area. Simulation Parameters exposes separate minimization, optional NVT, optional NPT, production, output/COM-removal, and execution-hardware controls; stages remain strictly serial and retain Martini 3-compatible non-bonded defaults.

4. Command-line interface

4.1 Discover commands and capabilities

```
gmxbuilder --help
gmxbuilder build --help
gmxbuilder serve --help
gmxbuilder martini3-bilayer --help
gmxbuilder martini3-solvent --help
gmxbuilder info --pdb protein.pdb
gmxbuilder list-ff
gmxbuilder list-water
gmxbuilder list-lipids
gmxbuilder lipid-library status
```

4.2 YAML atomistic build

```
system_name: membrane_system
output_dir: ./output
seed: 42

modules:
  input:
    pdb: ./protein.pdb
  forcefield:
    name: amber14sb
    lipid_ff: lipid21
    ligand_ff: none
    water_model: tip3p
  structure:
    pH: 7.0
    prepare_standard termini: true
  orient:
    method: ppm
  membrane:
    lipid_type: POPC
    box_padding: 2.0
  solvation:
    box_padding: 2.0
  ions:
    cation: NA
    anion: CL
    concentration: 0.15
    neutralize: true
    ion_method: random
  topology: {}
  simparams:
    schema_version: 2
  export:
    write_mdp: true
  execution_hardware:
    mode: thread-mpi
    cpu_threads: 8
    mpi_ranks: 2
    use_gpu: true
    gpu_count: 1
    gpu_ids: [0]
    gmx_command: gmx
```

```
gmxbuilder build --config build.yaml
gmxbuilder build --config build.yaml --output ./another-output
```

Every selected lipid and retained molecule must be valid for the complete force-field combination. CLI failures use non-zero exit status. Module-level configuration is validated; legacy top-level pipeline extension keys may be ignored for compatibility. Use the current schema rather than relying on unknown keys.

For YAML/CLI builds, execution hardware belongs to `export.execution_hardware`, not `simparams`. It changes the generated launcher and never changes the accepted molecular coordinates or MDP physics. HTTP Build requests use the separate `modules.execution` object shown by the installed OpenAPI schema.

4.3 Martini 3 example

```
gmxbuilder martini3-bilayer \  
--upper POPC:3 --upper CHOL:1 \  
--lower POPE:1 --lower POPG:1 \  
--lipids-per-leaflet 150 --padding 2 --salt 0.15 \  
--threads 8 --mpi-ranks 1 --gpu-ids 0 \  
--output ./martini-system
```

Use `--pdb protein.pdb` for a protein-bilayer system. For an aqueous system use `gmxbuilder martini3-solvent --pdb protein.pdb ....`

5. HTTP API

5.1 Discovery and error handling

When the service is running:

- Swagger UI: <http://127.0.0.1:7788/docs>
- OpenAPI schema: <http://127.0.0.1:7788/openapi.json>

Useful discovery endpoints:

```
GET /health
GET /api/hardware
GET /api/task-types
GET /api/options
```

Clients must check both the HTTP status and the JSON body. **4xx** denotes invalid input, unavailable capability, missing prerequisite, or expired Task; **5xx** denotes a server-side failure. Do not continue after an error response.

5.1.1 Managed asynchronous operations

Expensive uploads, Check operations, viewer preparation and Build may first return 202 with an **X-GMXBUILDER-Operation** header and an **operation_id** in JSON. This acknowledges admission, not success. Poll the operation until **response_ready** is true, then retrieve the original response and check its HTTP status before continuing. The browser performs this exchange automatically.

```
API=http://127.0.0.1:7788
curl -fsS "$API/api/operations/<operation-id>"
curl -sS "$API/api/operations/<operation-id>/result"
```

Apply this sequence to every request below. Obtain a Task ID from the completed upload result, and submit subsequent steps serially. HTTP 410 means expired or removed; 507 indicates unavailable storage. Managed installations retain tasks for 24 hours from creation by default; waiting, refresh and download do not extend that deadline. Save completed downloads. **GET /api/resource-policy** reports the actual configuration and whether isolation is active.

A compatibility query returning **503** with **code: missing_compatibility_data** means required force-field or checkpoint data is absent. Complete installation and rerun the input Check before continuing; this response does not permit Build.

5.2 Minimal Solvator sequence

```
API=http://127.0.0.1:7788

curl -sS -X POST "$API/api/upload-pdb" \
-F "file=@protein.pdb" \
-F "task_type=solvator"

TASK_ID=<returned-task-id>

curl -sS -X POST "$API/api/step/$TASK_ID/input" \
-H "Content-Type: application/json" -d '{"config":{}}'

curl -sS -X POST "$API/api/step/$TASK_ID/forcefield" \
-H "Content-Type: application/json" \
-d '{"config":{"name":"amber14sb","lipid_ff":"none","ligand_ff":"none","water_model":"tip3p"}}'

curl -sS -X POST "$API/api/step/$TASK_ID/structure" \
-H "Content-Type: application/json" \
-d '{"config":{"pH":7.0,"prepare_standard_termini":true}}'

curl -sS -X POST "$API/api/step/$TASK_ID/solvation" \
-H "Content-Type: application/json" \
-d '{"config":{"box_padding":2.0}}'

curl -sS -X POST "$API/api/step/$TASK_ID/ions" \
-H "Content-Type: application/json" \
-d '{"config":{"cation":"NA","anion":"CL","concentration":0.15,"neutralize":true,"ion_method":"random"}}'
```

Bilayer clients also execute [orient](#) and [membrane](#) before solvation. Inspect saved checkpoints with [GET /api/steps/{task_id}](#).

5.2.1 Final structure confirmation

After the ion step succeeds, retrieve and inspect the complete final system. The JSON contains displayed coordinates, components, box, [source_step](#) and [revision](#); the Web Final Structure Review can also display the checkpoint. Confirm the current revision you actually inspected, not a stale or unseen one.

```
curl -fsS --compressed "$API/api/step/$TASK_ID/ions/viewer.json" -o final-system.json
# Inspect the final structure represented by final-system.json before confirming.
curl -sS -X POST "$API/api/task/$TASK_ID/final-review" \
-H "Content-Type: application/json" \
-d '{"source_step":"ions","revision":"<reviewed-revision>"}'
```

Use [membrane](#) for a dry pure bilayer, [cg_system](#) for Martini 3, and [ions](#) for wet atomistic systems. An upstream change requires a new inspection and confirmation; Build rejects an unconfirmed current checkpoint.

5.3 Build, queue, and download

```
curl -sS -X POST "$API/api/build" \
-H "Content-Type: application/json" \
-d '{
  "task_id":"","TASK_ID",
  "task_type":"solvator",
  "system_name":"solution_system",
  "modules":{
    "simparams":{"schema_version":2},
    "export":{"write_mdp":true}
  }
}'

curl -sS "$API/api/build/$TASK_ID/queue-status"
curl -fL "$API/api/task/$TASK_ID/download" -o result.zip
```

Queue responses include position and an estimated start time. Save the Task ID before leaving the page. [GET /api/tasks](#) is an administrator endpoint and requires [X-Admin-Token](#).

6. Output package and execution

6.1 Package layout

```
README.txt
manifest.json
CITATIONS.json
run_md.sh          # wet systems with the relevant stages enabled
structure/
  input.gro
  input.pdb        # omitted when PDB format limits are exceeded
  index.ndx
topology/
  topol.top
  forcefield/      # atomistic force-field database
  <molecule>.itp
mdp/
  mini.mdp
  equili_<n>.mdp
  production.mdp or production_<n>.mdp
```

[topology/topol.top](#) is the authoritative include list. Martini parameters and protein ITPs use [topology/toppar/](#). All workflows include [manifest.json](#) and [CITATIONS.json](#); molecule-specific files and index groups vary with the system. Dry Pure Bilayer packages omit solvent, ions, MDP files and the run script.

The manifest records versions, input hashes, settings and output-file checksums. The original upload is not redistributed: retain the original input, configuration and matching parameter sources for reconstruction. Review the manifest before simulation. See [1.0.0 release files](#) for software download contents.

6.2 Run the generated package

```
unzip result.zip -d simulation
cd simulation
chmod +x run_md.sh
./run_md.sh
```

Read [README.txt](#) and inspect every [grompp](#) and [mdrun](#) message. Do not suppress warnings without understanding their physical and topology implications.

[README.txt](#) records the build seed and the velocity seed. The velocity seed is derived from the build seed and written into the MDP files as an explicit positive [gen-seed](#), rather than the runtime-generated [gen-seed = -1](#), so the initial velocities of a package can be reproduced. Two builds that differ only in their seed therefore produce different initial velocities, and repeating a build with the same seed reproduces them.

Every generated MDP file is validated after any advanced overrides have been applied, so the shipped file rather than the browser form is what is checked. The validation covers the integrator, the timestep against the constraint scheme in force, cutoff ordering, the CHARMM force-switch requirements, and the clearance between the solute and the periodic faces. A rejected protocol stops the build before any MDP file is written, so a partially refreshed set cannot be left behind.

7. Troubleshooting

7.1 A lipid is unavailable

The selected backend lacks either an exact topology or a conformer that passed the quality gates. Use the alternative shown by the UI or select another validated composition.

7.2 Bilayer orientation or packing looks wrong

Do not continue. Recheck protein orientation, leaflet assignment, conformer identity, composition, and the Step 5 quality report.

7.3 Automatic protein orientation is implausible

Treat the automatic score as a starting point. Use manual adjustment, Check again, and verify that the subsequent Viewer matches the saved orientation.

7.4 Water layers appear unequal

For membranes, padding is measured from the membrane interfaces. A protruding protein can make the visible solvent shape asymmetric without redefining the requested membrane-relative padding. Check numerical box and interface values.

7.5 Ions cluster or water is missing from the Viewer

Re-run Ion Check, verify the selected replacement method and exclusion radius, and inspect the exact complete-system Viewer. Do not proceed if coordinates do not match the reported counts.

7.6 ZIP lacks MDP files or run_md.sh

A dry bilayer intentionally omits them. For a solvated system, ensure that at least one simulation stage and MDP export were enabled, then rebuild from the confirmed checkpoint.

7.7 A Task cannot be resumed

Check the 32-character ID and the server retention period. Task-private custom lipids cannot be moved to another Task.

7.8 A pH is refused, or a build warns about the termini

See section 1.3. Inside pH 3.50 to 8.00 the build proceeds and states the populated fraction; outside it the build stops. Select free termini or supported caps according to the experimental construct and modeling objective. Truncation alone does not justify capping every chain, and changing molecular identity is not a general substitute for an unsupported terminal state.

7.9 A water model is refused as expert-unvalidated

The topology is present but the pair is not the force field's default and is not covered by regression tests. Either choose the force field's default water model, or set `allow_unvalidated_water_model: true` after establishing that the combination is appropriate for the system. The decision is recorded in the exported metadata either way.

7.10 A size limit is reported

Section 1.4 lists the bounds. The message names the value that exceeded its limit. For an upload limit raised through the environment, note that a value above the hard maximum is clamped to that maximum and the refusal appears in the service log; the effective limit is therefore not always the requested one.

7.11 Automatic installation cannot build GROMACS

Confirm that CMake, a C++17 compiler, Python [venv](#), sufficient disk space, and network access are available. To bypass a local CUDA-toolchain problem, retry with `GMXBUILDER_GROMACS_FORCE_CPU=1`. To use an administrator-provided compatible build, pass its `gmx` executable with `--gmx-bin`. Do not point the installer to GROMACS older than 2026.0, because the bundled Amber ff14SB port requires the newer preprocessing behavior.

8. Getting help

```
gmxbuilder --help
gmxbuilder list-ff
gmxbuilder list-water
gmxbuilder list-lipids
gmxbuilder lipid-library status
curl -fsS http://127.0.0.1:7788/health
```

When reporting a problem, include the GMXBUILDER version, workflow step and complete error text with a screenshot or log excerpt free of sensitive structure data. Share a Task ID only privately with a trusted administrator; do not include it in a public issue or screenshot.

Appendix A. Deployment configuration

1. Use a non-root service account. The default installation listens on [127.0.0.1:7788](#).
2. Unauthenticated non-loopback [local](#) / [trusted-lan](#) listeners require explicit unsafe opt-in and a protected private network.
3. Authenticated public hosting uses [GMXBUILDER_DEPLOYMENT_MODE=public](#) with strong Basic or Bearer credentials.
4. Anonymous public hosting uses [GMXBUILDER_DEPLOYMENT_MODE=public-anonymous](#), requires managed storage and computation isolation, and keeps [GMXBUILDER_ALLOW_UNSAFE_DEPLOYMENT=0](#).
5. Both public modes require explicit HTTPS [GMXBUILDER_CORS_ORIGINS](#) and exact [GMXBUILDER_TRUSTED_PROXIES](#). The proxy must replace untrusted forwarding headers. Set [FORWARDED_ALLOW_IPS=](#) empty so the application can validate the actual socket peer.
6. Configure a separate strong [GMXBUILDER_ADMIN_TOKEN](#). Keep Task IDs, operation IDs and credentials out of public logs.
7. Monitor [GET /health/ready](#); a reachable home page or successful [/health/live](#) does not establish writable quota storage.
8. Managed defaults are 16 GiB memory per operation, 2 GiB storage per task, 100 GiB total storage and 24 hours from task creation. Inspect [/api/resource-policy](#) for actual values. A plain development server does not claim production isolation.

See [anonymous resource setup](#) and [Web readiness and recovery](#) for installation, migration and maintenance. These Web operations do not start or authorize an offline lipid simulation queue.